

Handout: Einführung in die Programmiersprache C (Teil 1)

1. Grundlagen & Geschichte

- **Ursprung:** Anfang der 1970er Jahre von **Dennis Ritchie** bei Bell Labs entwickelt (Weiterentwicklung der Sprache B).
- **Zweck:** Ursprünglich für die Entwicklung des Betriebssystems **Unix**.
- **Bedeutung:**
 - Basis für viele moderne Sprachen (C++, Java, C#).
 - Standard für Betriebssysteme, Treiber und eingebettete Systeme.
 - **Vorteile:** Hohe Effizienz, schneller Ausführung, direkter Hardware- und Speicherzugriff.
- **Standardisierung:** 1989 als **ANSI C** normiert.

2. Programmstruktur & die `main()`-Funktion

Ein minimales C-Programm folgt diesem Aufbau:

```
#include <stdio.h> // Einbinden der Standard-Ein-/Ausgabe-Bibliothek

int main(int argc, char *argv[]) {
    printf("Hello, world!\n"); // Ausgabe auf das Terminal
    return 0;                 // Rückgabe an das Betriebssystem (0 = Erfolg)
}
```

- **#include <stdio.h>:** Header-Datei für Funktionen wie `printf()` und `scanf()`.
- **main():** Der definierte Startpunkt jedes Programms.
- **Rückgabewert:** Das Programm gibt einen Wert an den aufrufenden Prozess zurück. 0 signalisiert meist einen fehlerfreien Durchlauf.

Kommentare

- **Blockkommentar:** `/* Text */` (über mehrere Zeilen möglich).
- **Zeilenkommentar:** `// Text` (bis zum Zeilenende).

3. Variablen & Datentypen

Variablen-Regeln

- **Name:** Besteht aus [a-zA-Z0-9_], darf nicht mit einer Ziffer beginnen, ist *case-sensitive* (Groß-/Kleinschreibung wichtig) und darf keine Schlüsselwörter sein.
- **Deklaration:** Variablen müssen vor der Nutzung deklariert werden.
- **Initialisierung:** Gleichzeitige Deklaration und Zuweisung: `int x = 5;`

Wichtige Basistypen

Typ	Beschreibung	Typische Größe (64-Bit)
char	Einzelnes Zeichen (ASCII)	1 Byte
int	Ganzzahlen	4 Bytes
long long	Sehr große Ganzzahlen	8 Bytes
float	Fließkommazahl (mittlere Genauigkeit)	4 Bytes
double	Fließkommazahl (hohe Genauigkeit)	8 Bytes

Deklarations-Spezifizierer

- **const:** Wert ist nach der Initialisierung unveränderbar (*read-only*).
- **unsigned:** Nur positive Werte (erweitert den Bereich nach oben).
- **static:**
 - In Funktionen: Variable behält Wert zwischen Aufrufen.
 - Global: Sichtbarkeit auf die Datei beschränkt.

Scope (Gültigkeitsbereich)

- **Block Scope:** Variablen in { ... } sind nur dort sichtbar.
- **Globaler Scope:** Variablen außerhalb aller Funktionen sind in der ganzen Datei sichtbar.
- **Standardwerte:** Globale/statische Variablen sind initial 0; lokale Variablen haben einen **undefinierten** Startwert.

4. Operatoren

Arithmetische Operatoren

Operator	Bedeutung	Operator	Bedeutung
*	Multiplikation	+	Addition
/	Division	-	Subtraktion
%	Modulo (Rest)		

Präzedenz: *, /, % vor +, -. Klammern () überschreiben die Rangfolge.

Vergleichs- & Logische Operatoren

- **Vergleich:** == (gleich), != (ungleich), >, <, >=, <=
- **Logisch:**
 - && (**UND**): Wahr, wenn beide Seiten wahr sind.
 - || (**ODER**): Wahr, wenn mindestens eine Seite wahr ist.
 - ! (**NICHT**): Kehrt den Wahrheitswert um.
- **Lazy Evaluation (Short-circuiting):** Bei && wird nach einem 0 (falsch) nicht weiter geprüft; bei || nach einer 1 (wahr).

5. Zeichenrepräsentation & Steuerzeichen

Zeichen (char) werden intern als Ganzzahlen (ASCII) gespeichert (z. B. 'A' = 65).

Wichtige Steuerzeichen (Escape-Sequenzen):

- \n: Zeilenumbruch
- \t: Tabulator
- \\: Backslash
- \": Doppeltes Hochkomma
- \0: Nullbyte (Ende eines Strings)

6. Standardbibliothek `stdio.h`

`printf()` (Ausgabe)

Formatierte Ausgabe mit Platzhaltern:

- `%d / %i`: Ganzzahl (`int`)
- `%u`: Vorzeichenlose Ganzzahl (`unsigned int`)
- `%c`: Zeichen (`char`)
- `%s`: Zeichenkette (`String`)
- `%f`: Fließkommazahl (`float/double`)
- `%p`: Speicheradresse (`Pointer`)

`scanf()` (Eingabe)

Liest Werte von der Tastatur ein. **Wichtig:** Bei den meisten Typen muss das `&`-Zeichen (Adresse) vor die Variable gesetzt werden: `scanf("%d", &variable);`

7. Kontrollstrukturen (Verzweigungen)

Mit `if`, `else if` und `else` wird der Programmfluss gesteuert:

```
if (bedingung1) {  
    // Code, wenn bedingung1 wahr ist  
} else if (bedingung2) {  
    // Code, wenn bedingung1 falsch, aber bedingung2 wahr ist  
} else {  
    // Code, wenn alle obigen Bedingungen falsch sind  
}
```

Es wird immer nur der **erste** zutreffende Zweig ausgeführt.